音速并行：基于 Artix-7 的纯 RTL 音频一体化系统

摘要

本作品在 Xilinx Artix-7 XC7A100T 上以纯 RTL（未使用 HLS、软核或任何现成 IP）实现了一套音频一体化系统，覆盖“采集 → 均衡 → 频谱分析 → 可视化 → 交互”的完整闭环。

系统以 WM8960 编解码器为音频前端（48 kHz / 24 bit），采样数据同时进入两条并行流水线：一是 5 段定点 IIR 参量均衡器，逐段做 Q 格式与位宽预算，各段增益可在线独立调节并实时回放；二是 1024 点基 2 FFT 频谱分析引擎，仅一个蝶形单元配合双口 BRAM 时分复用，硬件开销不到全并行方案的 1/512，整帧处理仅占帧周期的 1.3%。频谱结果经对数频率分箱、峰值保持与线性衰减后驱动双路独立显示：LCD 480×272 作交互界面（8 按钮面板 + 8 行状态栏），HDMI 640×360 作展示大屏（80 根独立柱 + 波形），并从 SD 卡读取图片作为动态背景（LCD 1:1 满屏、HDMI 裁中心 320×180 后 2 倍整数放大）。

系统横跨四个时钟域，以格雷码异步 FIFO 与电平翻转握手完成跨域传递。所有参数可经板载按键或 UART ASCII 命令在一个时钟周期内生效。

整机占用 11 716 LUT（18.48%）、100.5 BRAM、26 DSP48，时序全部收敛（`clk_sys` +1.779 ns / `clk_hdmi` +4.028 ns），综合与实现 0 错误 0 警告，39 个 testbench 全部与 Python 定点黄金模型逐位对拍通过，并已通过 QSPI Flash 固化实现掉电自启动。

关键字：音频一体化　FPGA　纯 RTL　频谱可视化　跨时钟域

目录

1 作品概述 …… 4
1.1 选题背景与实际问题 …… 4
1.2 功能与特性 …… 5
1.3 主要技术特点及创新点 …… 7
1.4 主要性能指标 …… 8
1.5 设计流程 …… 9
2 系统组成及功能说明 …… 10
2.1 整体介绍 …… 10
2.2 硬件系统介绍 …… 11
2.2.1 硬件整体介绍 …… 11
2.2.2 电路各模块介绍 …… 12
2.3 软件系统介绍 …… 16
2.3.1 软件整体介绍 …… 16
2.3.2 软件各模块介绍 …… 17
3 完成情况及性能参数 …… 23
3.1 整体介绍 …… 23
3.2 工程成果 …… 24
3.3 特性成果 …… 25
4 总结 …… 27
4.1 大模型协作记录 …… 27
4.1.1 所用 AI 工具与使用环节 …… 27
4.1.2 主要提示方式 …… 27
4.1.3 自我纠错轨迹 …… 31
4.1.4 智能体工作流设计 …… 32
4.1.5 采纳、人工修改与核验 …… 33
4.2 技能包的提炼过程 …… 34
4.2.1 三级沉淀体系 …… 34
4.2.2 三个技能包的来源、验证与边界 …… 34
4.2.3 边界如何确定 …… 34

4.3 可扩展之处 …… 35
4.4 心得体会 …… 35
参考文献 …… 37
A 附录：复现说明 …… 38
1.1 环境要求 …… 38
1.2 从零开始的完整步骤 …… 38
1.2.1 步骤 1：获取代码 …… 38
1.2.2 步骤 2：工具链自检 …… 38
1.2.3 步骤 3：全量仿真（约 1 分 30 秒） …… 39
1.2.4 步骤 4：静态检查 …… 39
1.2.5 步骤 5：综合与实现（约 9~10 分钟） …… 39
1.2.6 步骤 6：准备 SD 卡（需要 sudo） …… 39
1.2.7 步骤 7：上板 …… 39
1.3 常见问题 …… 40

1 作品概述

1.1 选题背景与实际问题

音频可视化正处在一个快速扩张的市场中。根据 Business Research Insights 2026 年发布的《Audio Visualizer Market》报告，全球音频可视化市场规模在 2026 年估计为 27.4 亿美元，该报告预计到 2035 年将增长到 116.1 亿美元，对应 2026 至 2035 年 17.4% 的年复合增长率 [1]。该报告指出，增长的主要驱动力来自媒体消费体验的持续升级、现场演出与社交媒体内容对创意可视化的需求，以及交互式显示设备的普及；同时，实时音频可视化与 AI、增强现实技术的结合也成为最新的发展趋势。这意味着可视化不再只是播放器上的一个装饰性插件，而是逐渐成为音频产品的基础能力。在此背景下，本作品选择在 FPGA 上以纯 RTL 实现一套完整的音频可视化系统。

音频可视化设备在消费市场需求广泛，例如车载音响频谱显示、直播/演出调音台可视化、Hi-Fi 播放器界面等等。此外，在工业检测领域：如机械振动频谱监测、电机异响诊断、齿轮箱故障早期预警等案例中也经常依赖于频谱分析方法。虽然目前音频可视化技术已经非常成熟，但现有产品存在一个普遍的结构性矛盾，如表 1 所示：

表 1　现有方案的结构性矛盾

方案	成本	主要问题
PC + 声卡 + 上位机	低	延迟高（10~100 ms），功耗大，不适合嵌入式部署
MCU + CMSIS-DSP	低	算力不足，1024 点 FFT + 5 段 EQ + 双屏驱动无法同时满足
Zynq + PYNQ	中	依赖 HLS/Python 框架，底层时序不可控，启动秒级

核心矛盾在于“实时性”与“可调性”难以兼得。专用 ASIC 实时但不灵活，软件方案灵活但延迟大。本作品主要要解决的问题以及和目前主流方案的对比如下：

本作品要解决的三个实际问题

1. **确定性延迟**：从 ADC 采样到屏幕像素，延迟以时钟周期计，不受操作系统调度、缓存命中率影响；
2. **在线可调**：所有参数（EQ 各段增益、配色、刷新率、视图）可经按键或 UART 进行配置，可在一个时钟周期内生效；
3. **零 IP 依赖**：从 I2C 主机、I2S 从机、FFT 蝶形、存储器控制器、TMDS 编码器到 5:1 串行化，全部手写，可逐行审计。

表 2　与已有工作的对比

方向	代表做法	本作品的差异
FPGA 音频 EQ	Xilinx FIR/IIR Compiler IP	手写二阶节级联，逐段做 Q 格式与位宽预算
FPGA FFT	Xilinx FFT IP（AXI4-Stream）	手写基 2 DIT + 单蝶形时分复用，资源为全并行方案的 1/512
频谱可视化	固定柱数、PC 侧渲染	扫描式方案，无 framebuffer，关键配置全参数化
FPGA HDMI	现成 IP 或 VHDL 开源核	手写 TMDS 编码器 + OSERDESE2 原语
开发方法	传统的 Vivado 构建方法	大模型协作 + skill + tcl 脚本

1.2 功能与特性

本作品是一套从模拟前端到双路显示输出的完整闭环系统，而非单个算法模块的演示。系统的功能可分为四个层次，如表 3 所示。**音频前端**负责把模拟信号变成可处理的数字样本，**分析引擎**把时域样本变成频域柱高与波形，**可视化层**把抽象数据变成可看的图像，**交互与存储**则负责参数与素材的进出。

在功能清单之上，本作品有三个值得单独展开的设计选择，也体现了在实际工程中容易遇到的一些问题以及解决的方法：

（一）扫描式双路渲染：一份核心，两块屏

常规做法是先渲染到帧缓冲，再由扫描逻辑读出，但 640×360×16 bit 的帧缓冲需要 103 块 BRAM，加上其它模块更是直接达到了 98% 左右，而本器件只有 135 块，实测会导致布线拥塞、时序更是崩到 -1.09 ns、导致 `place_design` 构建失败。

本作品把渲染核心抽象为纯组合函数 $(x, y, \text{mode}) \to \text{RGB}$：

```
// 同一个模块，靠参数决定分辨率和是否画 UI
disp_top #(.HDISP(480), .VDISP(272), .WITH_UI(1)) u_disp_lcd  (...);  // 12.5 MHz
disp_top #(.HDISP(640), .VDISP(360), .WITH_UI(0)) u_disp_hdmi (...);  // 25 MHz
```

这样带来的好处主要有三重：1. LCD 侧零 framebuffer，只有 SD 图片需要存储；2. 换分辨率只改参数，不改逻辑；3. 两路的配色、柱高、波形天然一致，因为它们看的是同一份输入数据。

（二）数据柱数与显示柱数解耦

LCD 的像素宽为 480，使用 60 根柱则每柱 8 像素，可以整除；而 HDMI 的像素宽为 640，若也用 60 根柱则每柱 10.67 像素，无法整除，所以必须用乘法器，但是乘法器会恶化时序。

表 3　系统功能清单（按四个层次组织）

层次	功能	状态	说明
音频前端	话筒拾音	✓	板载 MEMS 麦克风（LINPUT1），48 kHz / 24 bit
	5 段参量均衡	✓	二阶节级联，各段 −10 ∼ +10 dB 独立可调，25 条系数在线重载
	双声道输出	✓	WM8960 DAC 双声道，数字音量 Q1.7
	SPK 静音	✓	送全 0 采样，`data_en` 不受影响，I2S 时序稳定不爆音
	模拟旁路关闭	✓	修 R45/R46 的 0 dB 旁路默认开启问题，否则 DSP 全程无效
分析引擎	1024 点 FFT	✓	基 2 DIT，单蝶形时分复用，2048 频点逐位可核
	幅度估计	✓	$\max + \frac{7}{16}\min$ 近似，误差 < 0.6 dB
	对数频率分箱	✓	按 Bark 尺度分箱，60 根（LCD）/ 80 根（HDMI）两套表
	峰值保持 + 衰减	✓	每帧线性衰减，柱高不会抖
可视化层	三种视图	✓	柱状频谱 / 极坐标频谱 / 动态波形，可在同一屏叠加
	扫描式渲染	✓	$(x, y, \text{mode}) \to$ RGB 纯组合函数，LCD 侧零 framebuffer
	彩虹配色	✓	128 级彩虹表，色相漂移速度 4 档可调
	波形触发	✓	迟滞触发 + 自适应门限（带上下限钳位）+ 刷新率可降
	双路输出	✓	LCD 480×272@12.5 MHz + HDMI 640×360@25 MHz，设置自动同步
交互与存储	8 按钮面板	✓	VIEW / SPK / HUE / GAIN / BG / DEMO / IN / SD
	8 行状态栏	✓	每行 10 字符，实时显示全部配置状态
	UART 命令通道	✓	115200 8N1，ASCII 命令改任意配置并回执
	SD 图片 + Flash	✓	3 张图循环（LCD 满屏 1:1 / HDMI 裁剪后 2 倍放大）；QSPI 掉电自启动

本作品把数据大小和显示柱数拆成两个独立参数（`NBARS` / `NBARS_DISP`），HDMI 侧改用 80 根柱可以整除，只需移位。更关键的是原本路径由 `generate` 的恒等分支保护：

```
generate
    if (NBARS_DISP == NBARS) begin : g_ident
        assign bbar = dbar[AW-1:0];              // 综合器看不到除法
    end else begin : g_scale
        assign bbar = (dbar * NBARS) / NBARS_DISP;
    end
endgenerate
```

（三）三个细节层面的设计

- **静音改用“送 0 数据”而不是“停 I2S”**：`data_en` 照常拉高，只是把左右声道数据清零。这样 WM8960 收到的是“合法的 0 采样数据”，链路不会以为对端掉线，避免由静音或取消静音而产生的瞬间爆音。
- **跨域握手用“电平翻转”而不是“单拍脉冲”**：`start` 若是单拍脉冲，跨到更慢的时钟域时可能被整个漏掉；改成翻转一个电平再双打取边沿，无论两侧时钟比多大都不会丢。
- **按键活动标志也用电平翻转**：面板的空闲自动收起计时器需要知道“用户刚才是否有动作”，同样不能用一个拍宽的脉冲跨域。

1.3 主要技术特点及创新点

1. 单蝶形 FFT 的极低资源占用

1024 点 FFT 若全并行需 512 个蝶形 × 4 乘法器 = **2048 个 DSP48**，超出器件总量（240 个）近 8 倍。而本作品用一个蝶形单元加双口 BRAM 进行时分复用：

表 4　FFT 架构方案对比（1024 点，48 kHz 采样）

方案	蝶形单元	DSP48	BRAM	整帧耗时	占帧周期
全并行	512	2048	0	1 拍	0.04%
4 路并行	128	512	0	7 680 拍	0.3%
单蝶形（采用）	**1**	**4**	**2**	**30 720 拍**	**1.3%**

WM8960 采样数据一帧周期大约为 46.9 ms，而 FFT 计算整帧耗时 30 720×20.83 ns = 0.64 ms，仅占 1.3%，余量为 75 倍。

2. 扫描式渲染，无帧缓冲

把渲染核心抽象为纯组合函数 $(x, y, \text{mode}) \rightarrow \text{RGB}$，扫描到哪个像素就算哪个像素。LCD 侧无需任何 framebuffer，一帧 130 560 个像素全部实时算出。

3. 零成本的裁剪与放大

HDMI 侧需要从 480×272 的全图中裁出中心 320×180 的区域，再放大 2 倍铺满 640×360。在实际操作中，可以通过两次移位相加从而避免使用到乘法器：

```
// 裁剪地址：dy*320 = dy*256 + dy*64 -> 两次移位相加，零 DSP
wire [18:0] crop_addr = (dy << 8) + (dy << 6) + dx;

```

```
// 2 倍放大：显示坐标右移 1 位 -> 一个移位，零算术
wire [XW-1:0] fb_sx = (fb_lx >> 1) + FB_CX[XW-1:0];
wire [YW-1:0] fb_sy = (fb_ly >> 1) + FB_CY[YW-1:0];
```

4. 不同尺寸双帧缓冲替代“双读口”

两个显示器时钟不同（12.5 MHz / 25 MHz）。实测表明 Vivado 的 BRAM 推断不支持“1 写 2 读跨时钟”，会静默退化成分布式 RAM（LUTRAM），触发 `DRC UTLZ-1` 错误（需 86 452 个 `RAMD64E`，仅 19 000 个可用），加 `(* ram_style = "block" *)` 进行约束也无效。

因此本作品改为两份不同尺寸的缓冲，即 LCD 侧直接读取 480×272 的全部图像数据，HDMI 读取对应的 320×180 范围内的裁剪区图像数据后进行 2 倍整数放大，这样一来 BRAM 从 116 块降到 84 块，节省一定资源的同时也避开了由静默推断而导致的错误。

1.4 主要性能指标

表 5　主要性能指标

类别	指标	值
资源	Slice LUTs	11 495 / 63 400（18.13%）
	Block RAM	100.5 / 135（74.44%）
	DSP48	26 / 240（10.83%）
	MMCM	2 / 6（33.33%）
时序	`clk_sys`（48 MHz）	WNS +**1.441 ns**
	`clk_hdmi`（25 MHz）	WNS +**5.879 ns**
	`clk_pix`（12.5 MHz）	WNS +**35.297 ns**
实时性	FFT 整帧耗时	0.64 ms（占帧周期 1.3%）
	端到端延迟	≈82 ms（80 ms 为 LCD 一帧扫描）
质量	综合/实现警告	**0 CRITICAL / 0 WARNING**
	静态检查	Verible 0 / Verilator 0
	仿真回归	**39 / 39 PASS**
显示	LCD / HDMI	480×272@12.5 MHz / 640×360@25 MHz
音频	采样 / 均衡	48 kHz / 24 bit / 5 段双二阶

1.5 设计流程

本作品采用的不是写完代码再补测试的传统流程，而是模型先行、逐位对拍、每轮全量回归的闭环流程，如表 1 所示。之所以这样做是因为 RTL 调试的最大成本不在于写，而是在于如何确认怎么写才是对的。尤其是在做定点算法时，肉眼根本看不出 1 LSB 的偏差。此外，在实际设计过程中，当遇到报错时，也需要区分是仿真报错还是综合报错，以采取相应的解决办法。

仿真红与综合红的不同处理方式

- **仿真红**：这是**逻辑存在错误**，可以定位后再继续修改。但有一个硬规则：同一问题连续修改 3 次都不对就必须停下来，回退到上一个稳定分支重新分析。在一个错误的分支上继续修改，只会把问题越搅越乱。
- **综合红**：这通常并非 bug，而是架构问题。例如 BRAM 占用到 98% 导致布线拥塞、进而产生 `clk_sys` 的时序违例 −1.09 ns、`place_design` 直接失败。所以这时候改代码几乎没用，必须从架构上进行修改，例如换存储方案、降低分辨率。

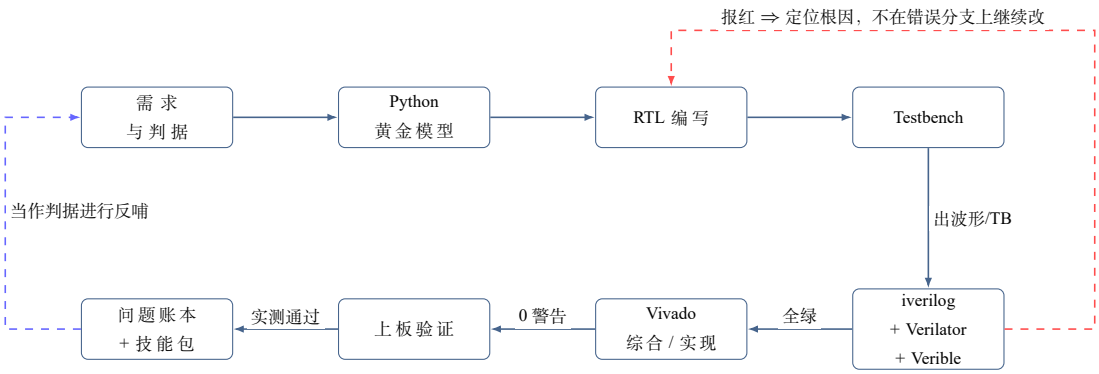


图 1　设计流程：模型先行、逐位对拍、每轮全量回归

2 系统组成及功能说明

2.1 整体介绍

系统的整体结构可以总结为一条从模拟前端到双路显示的单向数据流，加上横跨四个时钟域的配置分发网络。单向数据流确保了当显示侧即使因为 SD 卡读图卡住，音频回环也照常工作；配置分发网络则保证了任一时刻两块屏的配置一致性，因为它们读的是同一组寄存器。

整个系统按时钟域以及功能可以切成四块，主要分为音频处理区（`clk_sys` 48 MHz）、显示渲染区（`clk_pix` 12.5 MHz）、HDMI 输出区（`clk_hdmi` 25 MHz）、存储与交互区（`clk_sys`）。系统整体架构框图以及四块区域的功能总结分别如图 2 和表 6 所示。

首先，图中上半部分为音频处理区，主要包括音频采集和数据处理两部分。WM8960 将音频输入的模拟信号转换为数字信号，由 Audio-Top 模块读取和 DC-Block 模块处理后被分为两路，一路经过 FIR 低通滤波后当作波形数据送进 Disp-Top 模块进行显示，另一路则是被送进 FFT-Core 模块进行频域分析，计算后的数据再通过 spectrum 模块映射为柱状频谱送进 Display-Top 模块用于显示。

其次，图中左下角为存储与交互区，包含 SD 卡读取通道与三路交互输入。SD 卡的四根 SPI 信号经 SD-Control-Top 完成初始化与多块读取，取回的字节流由 SD-Img-Load 拼成像素写入 Frame-Buffer，再作为 DispTop 的背景源。交互方面，板载按键经 Key-UI（消抖 + 手势识别）产生光标与动作码，PC 端发送指令 UART-RX 送入 CMD-Proc 对 ASCII 命令进行解析，两者与触摸输入共同汇入 UI-Control 配置寄存器组进行统一管理，最后由 cdc-Sync 分发到各个时钟域，产生相应的动作。

最后，图中右下角为显示渲染区和 HDMI 输出区，两者共用同一份 Disp-Top 渲染核心，只是参数不同。在 Disp-Top 内部，LCD-Timing 产生扫描坐标，Bg-Src / Wave-Buf / Status-Line、Text-Buf、Font-ROM / UI-Layer / UI-Anim 五个图层模块各自接收同一组坐标独立计算，计算后的输出统一送进 Disp-Mix 与柱状频谱、极坐标频谱（Polar-Map + Rainbow-ROM）按优先级叠加，最终形成一帧像素流。该像素流分两路：一路直接驱动 LCD；另一路经 TMDS-Enc 编码与 TMDS-Out 串行化后输出 HDMI，同时 Frame-Buffer 中的 SD 图片也作为两路的背景参与合成。

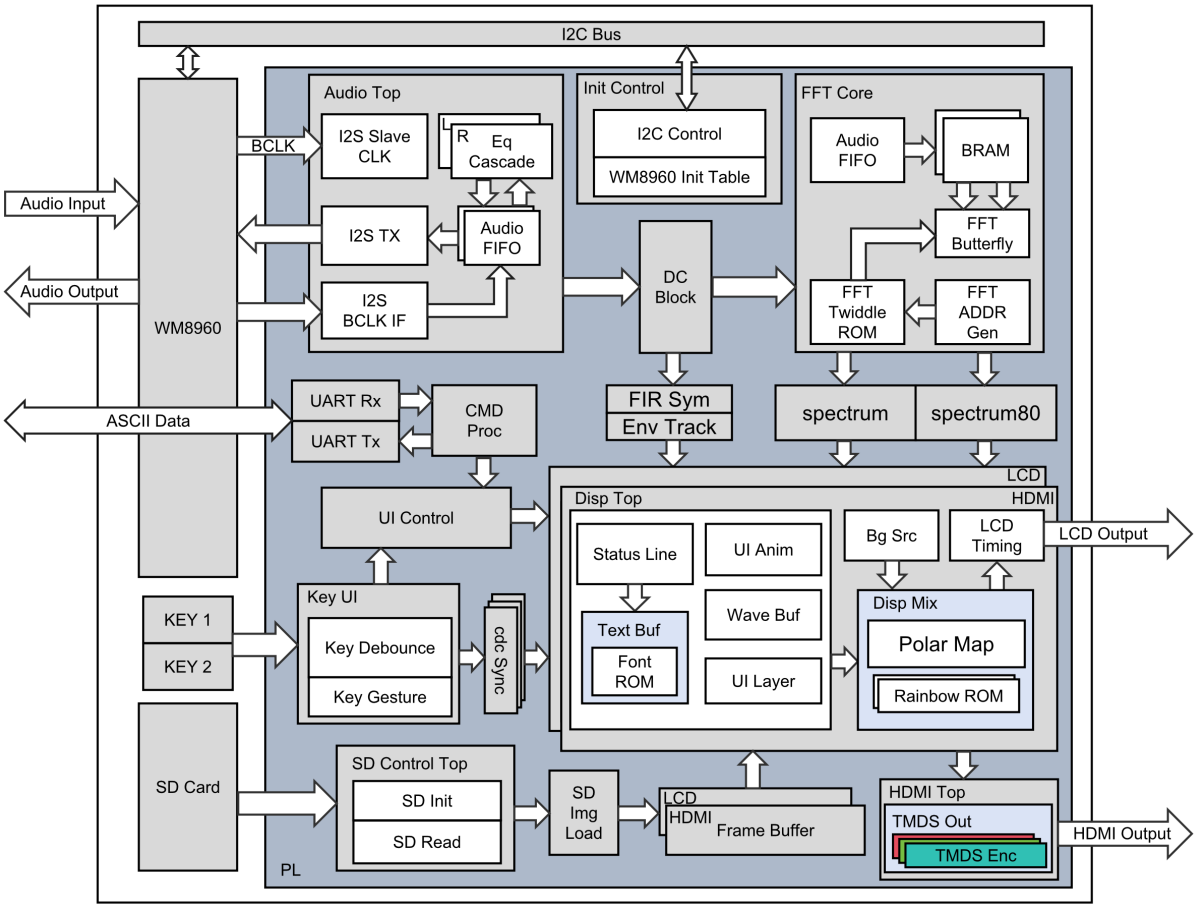


图 2　系统整体架构框图

表 6　四大区域的职责与关系

区域	时钟域	职责
音频处理区	`clk_sys` 48 MHz	I2S 从模式收发、5 段均衡、FFT、频谱分箱；产出柱谱和波形两类可视化数据
显示渲染区	`clk_pix` 12.5 MHz	LCD 扫描与像素合成；输入“坐标 + 柱高 + 波形 + 图片”，输出 RGB888 颜色数据
HDMI 输出区	`clk_hdmi` 25 MHz	接收 RGB888 数据并驱动 HDMI 屏幕进行显示
存储与交互区	`clk_sys` 48 MHz	SD 卡协议、图片搬帧、UART 命令、按键手势、配置寄存器组

2.2 硬件系统介绍

2.2.1 硬件整体介绍

本作品没有自制的 PCB，所有电路都是开发板与现成模块的组合，大部分功能实现都集中在 FPGA 内部。整机主要由 5 个物理部件构成：

1. **主控板**：璞致 PA-Starlite（XC7A100T-2FGG484I），板载 TypeC 口可同时用作供电与 JTAG 下载，一个 200 MHz 差分晶振与一路复位按键，以及两个 40 Pin 的扩展口，其余资源详见官方用户手册。
2. **音频模块**：WM8960 编解码器模块，插在主控板的 JM2 扩展口。该芯片同时支持 I2S 主模式和从模式，可通过 I2C 在上电时进行配置，当 FPGA 作为从设备时，接收由 WM8960 输出的 BCLK/LRCLK 分别作为位时钟和帧时钟。
3. **显示屏**：璞致 4.3 inch LCD，屏幕型号为 TIANMA-TM043NDH02，480×272 分辨率，插在 JM1 扩展口。由于该型号屏幕不带不带触摸层，所以虽然写了驱动代码但无法使用。
4. **HDMI 输出**：板载一个 HDMI 输出接口，但是没有 HDMI 发送芯片，所以所有 TMDS 编码与 5:1 串行化必须全部在 FPGA 内进行实现。
5. **存储**：板载一个 W25Q128JVSQ（128 MB QSPI Flash）可以用于电路固化，以及一个 microSD 卡座用于存储背景图片数据。

表 7　硬件组成

部件	型号	说明
主控板	璞致 PA-Starlite	XC7A100T-2FGG484I，200 MHz 差分晶振
音频模块	WM8960	编解码器，I2S 主模式，接 JM2
显示屏	璞致 4.3 inch LCD	480×272，40 pin，RGB888，接 JM1
视频输出	HDMI 输出	无板载发送芯片，由 FPGA 内部实现输出编码
存储	W25Q128JVSQ	128 Mb QSPI Flash，4 线
存储	microSD	SPI 模式，存图片素材
调试	CH340E	UART → USB，115200 8N1
下载	板载 Digilent JTAG	TypeC 接口

2.2.2 电路各模块介绍

由于本作品没有自行设计 PCB，全部电路均都来自开发板与现成模块，因此本节按外设逐块说明各接口的电气特性、关键信号与使用注意事项。整机外围共六类电路：两路 LED 指示与三个按键（复位、KEY1、KEY2）、UART 转 USB 电路、SD 卡座、HDMI 输出电路、LCD 接口以及 WM8960 音频模块。所有外设都挂在 FPGA 的 Bank 14 与 Bank 15/16 上，其中 Bank 14 的 Vcco 固定为 3.3 V，因此该 Bank 上的信号全部按 LVCMOS33 约束。

1. **LED 与 KEY**

板上提供两颗 LED 与三个按键。LED1（W22）与 LED2（Y22）均为高电平点亮，分别用作 WM8960 配置完成的指示与约 2.9 Hz 的心跳；复位按键（R14）、KEY1（W21）与 KEY2（Y21）都是按下为低，板上已自带拉电阻。三个按键都需要在 FPGA 内部做消抖，本工程取 20 ms；KEY1 与 KEY2 还要进一步区分单击与双击，判定窗取 300 ms。

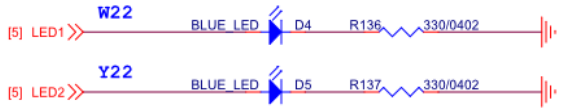


图 3　LED 电路原理图

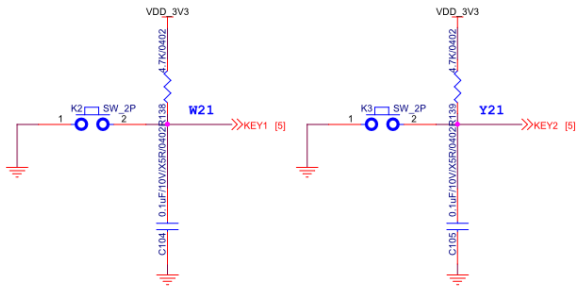


图 4　KEY 电路原理图

2. **UART 转 USB**

板载一颗 CH340E 用于 USB 到串口的转换，FPGA 侧只需两根信号：UART_TX（P15，FPGA → PC）与 UART_RX（P14，PC → FPGA），均为 3.3 V 电平。本工程用 115200 波特率、8 位数据、无校验、1 位停止位，作为配置命令通道使用，上电后会主动打印一次横幅，此后可接收 ASCII 命令并回执。

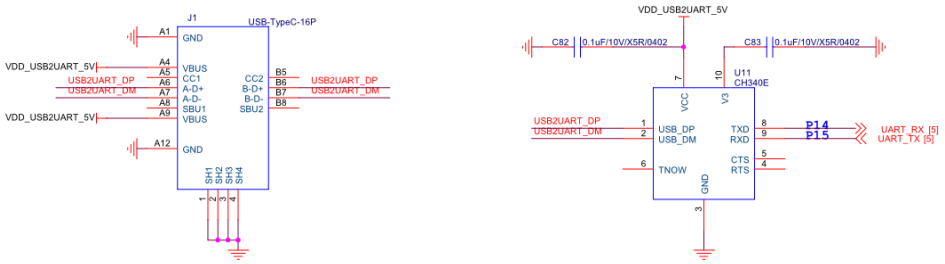


图 5　UART 转 USB 电路原理图

3. **SD Card**

SD 卡座有六根信号线（CLK、CMD、DATA0、DATA1、DATA2、DATA3），全部位于 Bank 14。本工程只用 SPI 模式，因此实际使用四根：SD_CLK（AA20）作为时

钟，SD_CMD（AB21）作为主机输出 MOSI，SD_DATA0（AB18）作为卡片输出 MISO，SD_DATA3（AA21）作为片选 CS。DATA1 与 DATA2 在 SPI 模式下不使用。

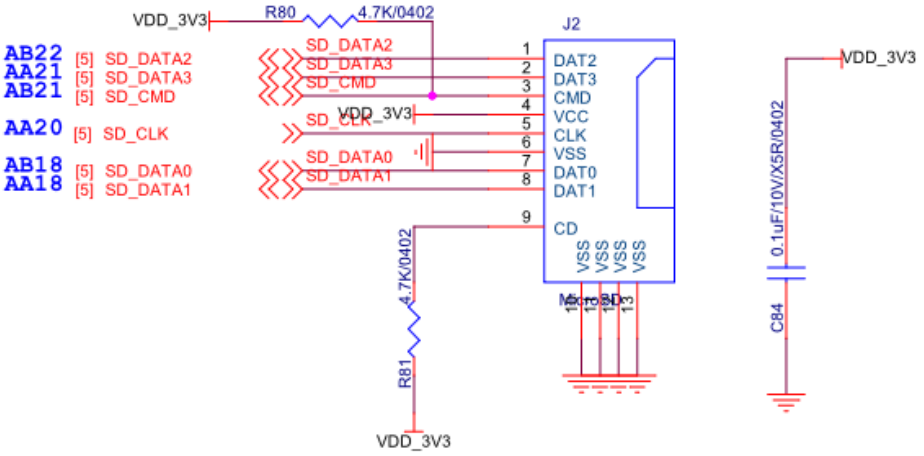


图 6　SD Card 电路原理图

4. **HDMI 输出**

HDMI 部分由四对差分信号与三个控制信号组成。四对差分分别是时钟对(Y18/Y19)、数据 0 对（V18/V19，蓝色通道）、数据 1 对（AA19/AB20，绿色通道）与数据 2 对(V17/W17，红色通道)，全部按 TMDS_33 标准约束。控制信号包括 I2C 的 DDC 通道（U20/T21）、热插拔检测 HPD（V20）与输出使能 HDMI_OUT_EN（V22）。此外还有一个值得注意的细节，HDMI_OUT_EN 引脚被接到 NC7SZ125 缓冲器的输出使能脚，低电平有效，若该脚浮空导致为高电平，屏幕会表现为无信号，很容易被误判成 TMDS 编码写错，所以代码中需要固定驱动为低电平。

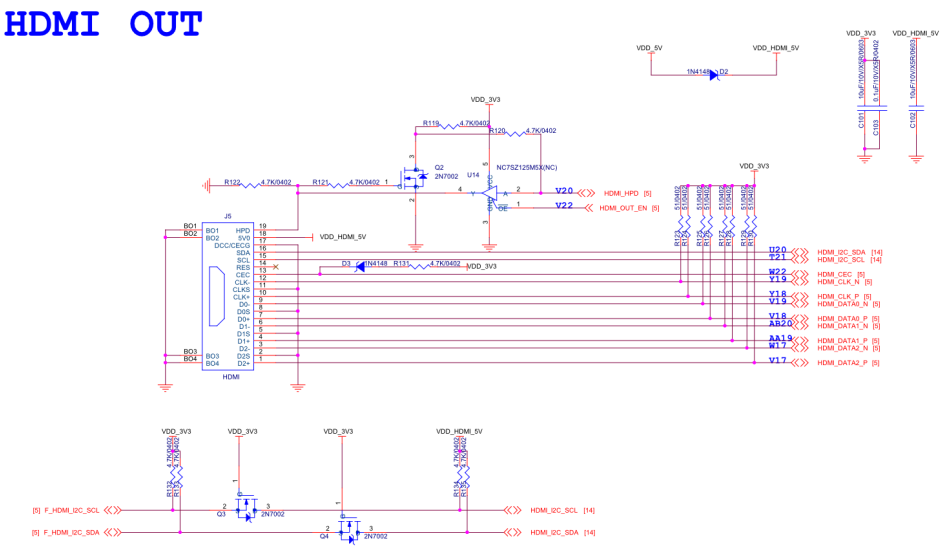


图 7　HDMI 输出电路原理图

5. **LCD 屏幕**

LCD 通过 40 pin 排线接在 JM1 扩展口，采用 RGB888 并行接口。除 24 根颜色信号外，还需要行同步 HS（C18）、场同步 VS（E18）与像素时钟 DCLK（F18），三者分别对应 JM1 的 29、32、30 脚。图中给出的 40P 座与 JM1 是 1:1 对应关系，对标签时需要先按排针的奇偶列分开，再按上下顺序读取，否则极易把相邻信号写反。

本工程未使用 LCD 的 DE 信号，而是靠 HS 与 VS 直接驱动该模组，实测可正常出图。屏幕模组型号为 TIANMA-TM043NDH02，不带触摸层，因此触摸屏驱动虽然已写好但未接入实际功能。

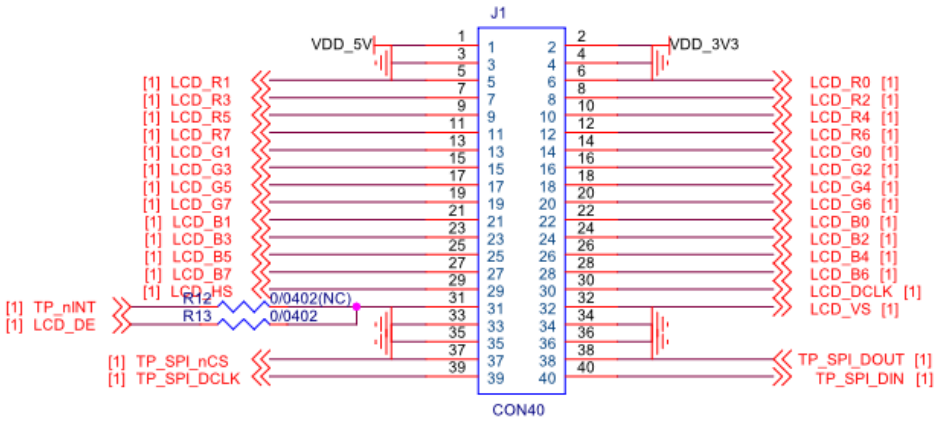


图 8　LCD 电路原理图

6. **WM8960 音频模块**

WM8960 音频模块通过杜邦线接到 JM2 扩展口，共六根信号。I2S 部分包括位时钟 BCLK（J20，3.072 MHz）、帧时钟 LRCLK（K18，48 kHz）、ADCDAT（H20，模块 → FPGA）与 DACDAT（G20，FPGA → 模块）；配置部分包括 I2C 的 SCL（K21）与 SDA（K22），全部按 LVCMOS33 约束。

关键点在于时钟方向，WM8960 的数字音频接口时钟从自身 SYSCLK 派生，手册要求 MCLK 与 BCLK、LRCLK 同源，因此该模块必然是 I2S 主设备，FPGA 只能作从设备。这意味着 BCLK 与 LRCLK 对 FPGA 而言是异步输入，本工程不把它们当作全局时钟，而是在 48 MHz 域内过采样恢复出边沿选通信号，从而少一个时钟域，也避开了时钟脚抖动的问题。

另外，该模块上电后必须通过 I2C 写入寄存器才能出声。其中模拟旁路的设置尤其关键，若旁路默认开启，音频信号会完全绕开 DSP 处理链，表现为静音功能失效、均衡器无效且只有单边喇叭出声。

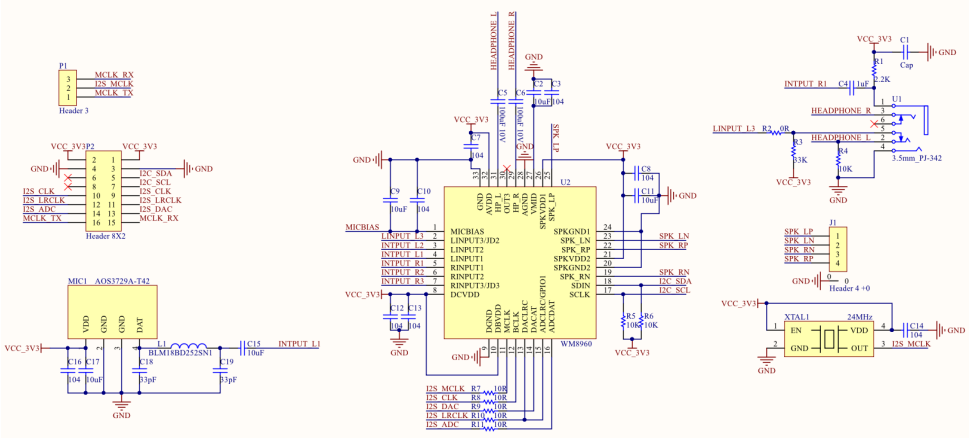


图 9　WM8960 电路原理图

2.3 软件系统介绍

2.3.1 软件整体介绍

本作品的软件部分主要分为两类：PC 端离线脚本和基于 Verilog 的纯 RTL 设计代码。其中，PC 端脚本主要用于生成 RTL 需要的一些表与图片数据，RTL 代码用于设计相应的电路并生成比特流烧录进 FPGA，全部 RTL 代码都是 Verilog-2001 标准，未使用 HLS 或相关 IP 核。每个独立脚本的作用见表 8，整个工程的文件见图 10。

表 8　PC 端离线脚本（Python）

脚本	职责 / 产物
`golden/dsp.py`	滤波器系数定点量化与误差评估 → `eq_coeff_rom.v`
`golden/fft_dsp.py`	FFT 旋转因子生成 + 定点误差分析 → `fft_twiddle_rom.v`
`golden/gen_spectrum_map.py`	对数频率分箱表 → `spectrum_map.v` / `spectrum_map80.v`
`golden/gen_font.py`	字模提取 → `font_rom.v`
`golden/gen_wm8960_table.py`	WM8960 寄存器表 → `WM8960_init_table.v`
`sdcard/mkimage.py`	BMP/PNG/JPG → RGB565 裸扇区
`sim/run_iv.sh`	39 个 TB 的并行回归（`JOBS=nproc`）
`vivado/build.tcl`	综合/实现/比特流（支持 `SYNTH_ONLY` / `IMPL_ONLY`）
`vivado/program.tcl`	SRAM 下载 / QSPI Flash 固化



图 10　工程文件结构

2.3.2 软件各模块介绍

四个主要模块的接口设计大致如下表所示：

表 9　四个主要模块的分工

模块	时钟域	输入	输出
`audio_top`	`clk_sys` 48 MHz	I2S 三根信号、增益档位	I2S 一根信号（DACDAT）、FFT 抽头
`fft_core`	`clk_sys` 48 MHz	24 bit 时域样本	512 个频点的幅度
`disp_top`	`clk_pix` / `clk_hdmi`	柱高、波形、图片、配置	一帧 RGB888 像素流
SD 卡通道	`clk_sys` 48 MHz	启动脉冲	两份帧缓冲的写口

1. 音频链路（audio_top）

模块对外只有三组信号。第一组是 I2S 物理接口，包括位时钟 bclk、帧时钟 lrclk、串行输入 sdin 与串行输出 sdout，其中前三个是输入，因为 WM8960 是本系统的 I2S 主设备。第二组是控制接口，包括 5 段 × 5 bit 的增益档位 band_gain、重载触发 cfg_load、旁路使能 bypass，以及反馈状态 cfg_busy 与 running。第三组是观测接口，dbg_rx_l/r/rx_valid 输出的是均衡之前的原始采样，直接送给 fft_core 进行下一步处理。该模块的接口定义如下：

```
module audio_top #(
    parameter integer DW    = 24,     // 样本位宽 Q1.23
    parameter integer NSECT = 5       // 均衡段数
) (
    input  wire clk,                  // clk_sys = 48 MHz
    input  wire rst_n,
    input  wire mute,                 // 1 = 静音（送全 0 采样）

    // I2S 物理接口：从模式，bclk/lrclk 由 WM8960 提供
    input  wire bclk,
    input  wire lrclk,
    input  wire sdin,                 // WM8960 ADCDAT -> FPGA
    output wire sdout,                // FPGA -> WM8960 DACDAT

    // 控制接口（必须已同步到 clk 域）
    input  wire [NSECT*5-1:0] band_gain,  // 每段 5 bit：0..20 对应 -10..+10 dB
    input  wire               cfg_load,   // 拉高即触发系数重载
    input  wire               bypass,     // 1 = 直通
    output wire               cfg_busy,
    output wire               running,

    // 观测接口：均衡之前的原始采样，送 fft_core
    output wire signed [DW-1:0] dbg_rx_l,
    output wire signed [DW-1:0] dbg_rx_r,
    output wire                 dbg_rx_valid
);
```

其中，cfg_load 的作用是把 eq_coeff_rom 里对应的系数搬进两个均衡器。装载过程由一个小状态机控制，共 25 次写操作（5 段 × 5 个系数），期间 cfg_busy 拉高、running 拉低，音频流水暂停弹出新样本，因此调整增益不会在听感上产生跳变。

2. FFT 引擎（fft_core）

该模块的接口定义如下：

```
module fft_core #(
    parameter integer N          = 1024,   // 点数
    parameter integer LOG2N      = 10,
    parameter integer DW         = 24,     // 数据位宽 Q1.23
    parameter integer TW         = 16,     // 旋转因子位宽 Q1.15
    parameter integer FIFO_DEPTH = 64
) (
```

```
    input  wire clk,                        // clk_sys = 48 MHz
    input  wire rst_n,

    // 输入样本流（实数）
    input  wire signed [DW-1:0] x_in,
    input  wire                 x_valid,

    // 输出幅度谱（前 N/2 个频点）
    output reg  [LOG2N-2:0]     y_index,    // 频点序号 0..511
    output reg  [DW:0]          y_mag,      // 幅度，多一位防溢出
    output reg                  y_valid,

    output wire                 busy,       // 1 = 正在运算
    output reg  [7:0]           drop_cnt    // 输入 FIFO 溢出计数，应恒为 0
);
```

输入端是标准的流接口，`x_in` 配 `x_valid` 送入实数样本；输出端给出频点序号 `y_index` 与对应幅度 `y_mag`。`drop_cnt` 是一个自检信号，如果输入速率超过处理能力导致 FIFO 溢出，它会计数，正常运行时必须恒为 0。

数据流设计：模块的内部只有一个蝶形单元，因此 1024 点所需的总蝶形数 $10\times512=5120$ 只能串行完成，状态机分三段：

1. `S_IDLE`：从输入 FIFO 连续取出 1024 个样本，按位反转序写入工作 BRAM。位反转是 DIT 结构的要求，如果输入按自然序存放，蝶形的两个输入就对不上。这一步不会额外占用时间。
2. `S_RUN`：10 级循环，每级 512 个蝶形。地址发生器同时产出三个量，分别为蝶形两点地址 `p` 与 `q`、以及旋转因子索引 `tw_idx`。前者送入 BRAM 的 A/B 读地址，后者送入旋转因子 ROM。蝶形从 BRAM 读出两个复数点，与旋转因子做一次复数乘加，再把结果写回同一对地址，形成读、算、写回的反馈环。
3. `S_OUT`：按自然序读出 512 个频点。

定点格式设计：这是 FFT 实现中最需要仔细处理的部分，本设计采用表 10 的位宽体系。

表 10　FFT 的定点格式与位宽推导

量	格式	位宽	推导依据
输入数据	Q1.23	24	与 I2S 24 bit 采样对齐，满幅范围 $[-1,1)$
旋转因子	Q1.15	16	$\lvert\cos\rvert,\lvert\sin\rvert\le1$，15 位小数精度约 3×10^{-5}
乘积	Q2.38	40	$\lvert a\cdot W\rvert\le2^{23}\times2^{15}=2^{38}$，加符号位共 40 位
累加	—	42	两个乘积相加减最坏 2^{39}，再留 1 位给舍入
蝶形输出	Q1.23	25	蝶形的 $a\pm t$ 幅度可能翻倍，必须多一位

3. 显示渲染（disp_top）

该模块的接口定义如下（参数较多，此处只列出与数据流相关的部分）：

```
module disp_top #(
    parameter integer HDISP      = 480,   // 水平有效像素
    parameter integer VDISP      = 272,   // 垂直有效行
    parameter integer NBARS      = 60,    // 数据柱数
    parameter integer NBARS_DISP = 60,    // 显示柱数（HDMI 用 80）
    parameter integer FB_SW      = 320,   // 帧缓冲存储宽
    parameter integer FB_W       = 320,   // 图片显示宽（放大后）
    parameter integer SHIFT      = 0,     // 放大倍数 log2
    parameter integer WITH_UI    = 1,     // 是否画面板
    parameter integer WITH_TEXT  = 1,     // 是否画状态栏
    parameter integer WAVE_AW    = 10     // 波形缓冲地址宽
) (
    //----------------- 音频域（clk_sys）-----------------
    input  wire                clk_sys,
    input  wire                rst_sys_n,
    input  wire                spec_wr,    // = spectrum.frame_done
    input  wire [NBARS*HW-1:0] spec_din,   // 柱高

    //----------------- 显示域（clk_pix）-----------------
    input  wire                clk_pix,
    input  wire                rst_pix_n,
    input  wire [3:0]          ui_mode,    // 背景模式
    input  wire [2:0]          ui_view,    // 视图使能
    input  wire [7:0]          ui_hue_spd, // 色相滚动速度

    //----------------- 波形（clk_sys 域）-----------------
    input  wire signed [23:0]  wave_din,   // = audio_top.rx_l
    input  wire                wave_we,
    input  wire signed [23:0]  wave_th,    // 自适应触发门限

    //----------------- SD 图片背景 -----------------
    input  wire                cfg_bg_src, // 1 = 背景取帧缓冲
    input  wire [15:0]         fb_rgb,
    output wire [18:0]         fb_addr,

    //----------------- 按键 -----------------
    input  wire                panel_tog,  // 面板开/关
    input  wire                key_act,    // 按键活动
    input  wire [3:0]          ui_sel,     // 光标位置

    //----------------- 输出 -----------------
    output wire [23:0]         lcd_rgb,
    output wire                lcd_hs,
    output wire                lcd_vs,
    output wire                lcd_de,
    output wire                lcd_clk
);
```

模块的核心输入是扫描坐标 x 与 y（由内部的 lcd_timing 产生），所有渲染子模块都接收同一组坐标，各自独立判断这个像素该不该自己画、该画什么颜色，相互之间没有任何数据传递。除坐标外还有四组输入：柱高 spec_din、波形 wave_din 与自适应

门限 wave_th、帧缓冲读出的 fb_rgb，以及配置寄存器组 ui_*。输出是一帧 RGB888 像素流，配合 HS、VS、DE 与像素时钟。

数据流设计。lcd_timing 产生坐标与同步信号，随后分出五条支路：bg_src 生成程序化背景，wave_buf 取出波形值，status_line 把配置状态写进 text_buf（字模取自 font_rom），ui_layer 绘制按钮面板，ui_anim 负责面板展开与色相过渡的插值。五条支路的输出全部进入 disp_mix，与柱状频谱、极坐标频谱按优先级叠加。叠加顺序从低到高为：背景、柱谱、极坐标、波形、状态栏文字、按钮面板。

高级效果设计。渲染部分的观感主要由三个效果决定，它们的实现思路与生成 128 级彩虹表的做法一致，即先在 PC 端算出数据表，再让硬件按地址读表即可。

1. **彩虹配色表**。颜色由 Python 脚本 gen_rainbow.py 通过 colorsys.hls_to_rgb 生成 128 级色相环，再取 0.85 的饱和度、0.6 的亮度，将结果输出为 rainbow_rom.v 里中。因此 RTL 侧只需输入 7 bit 地址即可得到 RGB 值。
2. **滚动色相漂移**。为了让画面产生“流动”的感觉，色相地址上叠加了一个随时间递增的偏移量 hue_off。递增节奏由 hue_spd 控制，实现方式是通过位掩码：tick_mask = (1 << (8 - hue_spd)) - 1。此后每帧对 frame_cnt 加一，当 frame_cnt 的低若干位全为 1 时 hue_off 加一。这样 7 档速度分别对应每 1、2、4 …128 帧漂移一级，无需任何除法器。这里有一个容易写错的地方：条件必须写成位与的结果全 1，若直接写成相等判断，则 256 帧才命中一次，色相几乎不动。
3. **波形自适应门限**。波形的稳定显示依赖触发，而固定门限在安静时完全无法触发。本设计用 env_track 跟踪音频包络并给出自适应门限，再在 disp_top 内部把它钳位在 32768 到 131072 之间。下限保证安静时也能正常触发，上限防止大音量时门限过高而产生多条波形重叠的假象。

4. SD 卡图片通道

该通道的接口分为两级。底层是 sd_ctrl_top，它对外暴露 SPI 物理引脚与一字节一拍的读数据流，接口的定义如下：

```
module sd_ctrl_top #(
    parameter integer CLK_HZ   = 48_000_000,
    parameter integer FAST_DIV = 4,        // 高速档 48M/4 = 12 MHz
    parameter integer INIT_HZ  = 200_000,  // 初始化阶段 SPI 时钟
    parameter integer BLOCKS   = 510       // 一次读多少块
) (
    input  wire        clk,                // clk_sys
    input  wire        rst_n,

    // SD 卡 SPI 引脚
    output wire        sd_clk,
    output reg         sd_cs,
    output reg         sd_mosi,
    input  wire        sd_miso,
```

```
    // 用户接口
    input  wire        rd_start,           // 单拍脉冲：开始读
    input  wire [31:0] rd_sec_addr,        // 起始扇区
    output wire        rd_busy,
    output wire        rd_valid,           // 单拍，rd_data 有效
    output wire [7:0]  rd_data,
    output wire        init_done
);
```

上层是 sd_img_load，把字节流拼成 16 bit 像素并同时写两份帧缓冲，整个过程大致分七步：

1. 首先将时钟切换到 400 kHz。SD 卡上电后必须以此速率完成初始化，超过则不响应。
2. 发送 CMD0，让卡进入 SPI 模式。
3. 发送 CMD8 查询电压范围，返回 R7 确认卡为 v2.0 及以上。
4. 循环发送 CMD55 与 ACMD41，直到 R1 的第 0 位变为 0。这一步必须支持重入，因为切换输入源时会重新初始化。
5. 时钟切回 25 MHz。
6. 发送 CMD18 连续读取 510 块。每块 512 字节，合计 261120 字节，正好是一帧 480×272×2 的数据量。一块读完后不回发命令，直接等待下一个数据令牌。
7. sd_img_load 拼像素并写帧缓冲，写满一帧后拉高 done。这里必须用 done 做门控停止接收，否则超出一帧的字节会反复写最后一个地址，把画面右下角踩坏。

```
module sd_img_load #(
    parameter integer W  = 480,
    parameter integer H  = 272,
    parameter integer EN_CROP = 1,   // 是否同时写裁剪缓冲
    parameter integer CX = 80,       // 裁剪区左上角
    parameter integer CY = 46,
    parameter integer CW = 320,
    parameter integer CH = 180
) (
    input  wire        clk,
    input  wire        rst_n,
    input  wire        start,         // 开始新一帧
    input  wire        byte_valid,    // 来自 sd_ctrl_top 的 rd_valid
    input  wire [7:0]  byte_data,

    output reg         wr_en,         // 第一份：全图
    output reg  [18:0] wr_addr,
    output reg  [15:0] wr_data,

    output reg         wr2_en,        // 第二份：仅裁剪区
    output reg  [18:0] wr2_addr,

    output reg         done,          // 一帧写满
    output reg  [18:0] pix_cnt
);
```

3 完成情况及性能参数

3.1 整体介绍

最终交付的是一个**可以独立工作、掉电重启后自动恢复**的完整系统：上电后 FPGA 从 QSPI Flash 加载比特流，MMCM 锁定后开始向 WM8960 发送 I2C 配置，配置完成后音频链路开始工作；用户对着麦克风说话，1 秒内 LCD 与 HDMI 两块屏就同时出现跳动的频谱。整个启动过程不需要任何上位机参与。整个系统主要从三个层面进行验证：

表 11　三层验证体系

层次	手段	覆盖什么
逻辑层	39 个 testbench，与 Python 定点模型逐位对拍	数值正确性（1 LSB 级），协议行为（SD/I2C/I2S/TMDS）
实现层	Vivado 综合/实现 + DRC + 时序报告	资源是否够、时序是否收敛、有无组合环/多重驱动
板级层	13 项人工验收清单（见表 12）	真实信号链、显示观感、交互响应

表 12　上板验收清单（全部实测通过）

#	检查项	实测结果
1	仿真回归 `run_iv.sh`	**39/39 PASS**
2	Verible / Verilator	0 违规 / 0 意外警告
3	Vivado 综合 + 实现	**0 CRITICAL / 0 WARNING**
4	`led[1]` 心跳	约 2.9 Hz 闪烁
5	`led[0]`	常亮（WM8960 配置完成）
6	LCD 程序化背景	网格 + 渐变正常显示
7	按 `BG` 键	程序化背景 / SD 图片切换
8	按 `SD` 键	3 张图片循环切换
9	对麦克风说话	频谱柱跳动、波形变化
10	按 `SPK` 键	喇叭静音/取消
11	HDMI 显示器	640×360 有画面（80 柱 + 波形 + 图片）
12	UART 115200	上电打横幅；发 `V` 切视图
13	QSPI Flash 固化	掉电后拔 JTAG 可自启动

最终实现效果如图 11 所示。

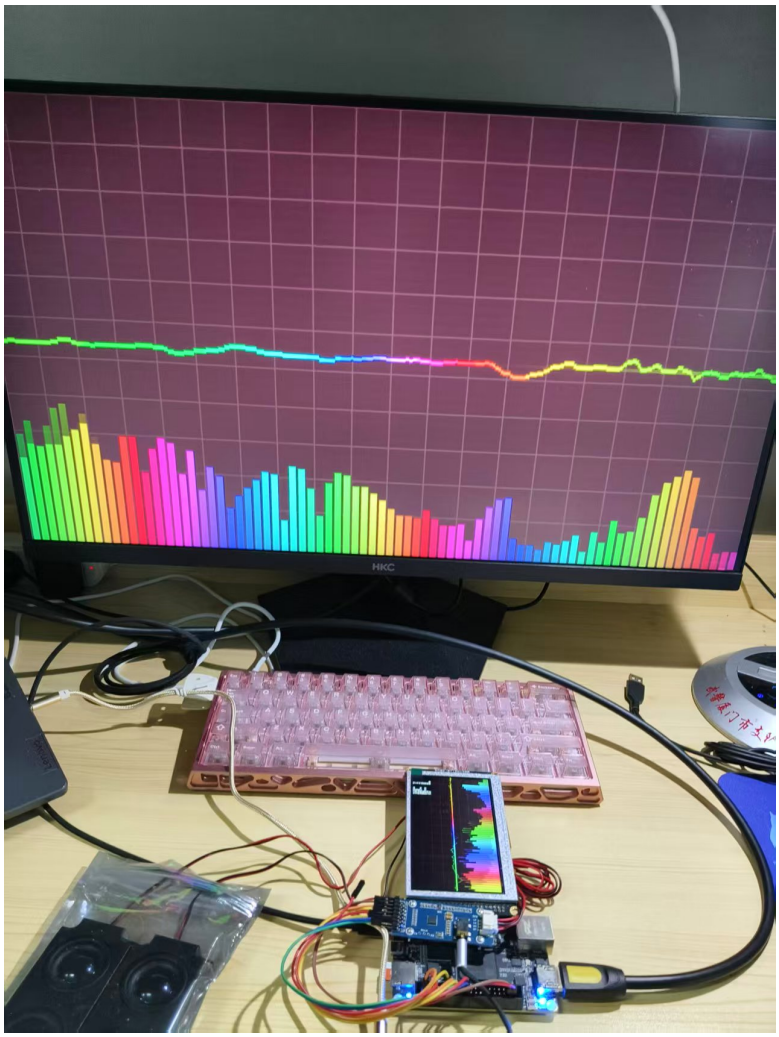


图 11　系统实物照片（斜 45°）

3.2 工程成果

本作品以开发板为核心，未做独立外壳，所有工作集中在 FPGA 内部，机械部分只需保证各模块能够稳插接即可。实物连接关系如下：

表 13　物理连接清单

接口	连接的部件	注意事项
JM1（扩展口）	璞致 4.3 inch LCD	断电插拔，避免损坏屏幕
JM2（扩展口）	WM8960 音频模块	由该板提供 BCLK/LRCLK（I2S 主）
HDMI 输出口	HDMI 线	板上无发送芯片，另一端接到 HDMI 屏幕
microSD 卡座	SD 卡	断电时插拔 SD 卡，避免损坏卡
JTAG 口	Type-C 线	系统供电 + 下载调试
UART 串口	Type-C 线	另一端接到 PC 上位机

系统上电顺序：插好所有模块 → 插 SD 卡 → 接 TypeC → 等 `led[1]` 开始闪烁（说明 MMCM 已锁定）。尽量不要在通电状态下插拔 LCD 或 SD 卡，避免造成器件损坏。

此外，可以看到两块屏的显示有所差异，这是刻意而为之的：LCD 小屏作为交互和预览界面，HDMI 大屏作为展示界面，只保留了最基本的视觉元素（柱谱 + 波形 + 图片背景）。

UART 通道主要用作开发期间进行在线调试，所有 LCD 面板上能点的功能，都有对应的 ASCII 命令（`V` 切视图、`A` 调增益、`B` 切背景…），并且每条命令都有回执，方便进行开发。

表 14　双路显示的定位差异

项	LCD 480×272	HDMI 640×360
定位	**交互界面**	**展示大屏**
分辨率	480×272 @ 60 Hz	640×360 @ 60 Hz
UI 层	**有**（8 按钮面板 + 8 行状态栏）	**无**（`WITH_UI=0`）
柱数	60 根（每柱 8 像素）	80 根（每柱 8 像素）
波形	有（×16 增益）	有（×16，幅度 80）
图片背景	480×272 全图 1:1 满屏	320×180 裁剪后 2× 放大
资源	`disp_top` 第 1 份实例	`disp_top` 第 2 份实例 + TMDS

3.3 特性成果

1. 优化过程与前后对比

本作品经历 5 个主要优化阶段：

表 15　五个优化阶段的量化收益

阶段	优化内容	关键指标变化
1	同步复位替代异步复位	`DPOR-1`: 11→**0**；WNS: +1.117→**+2.156 ns**；LUT: 6802→**5648**
2	FFT 全并行 → 单蝶形时分复用	DSP: 2048→**4**（1/512）；BRAM: 0→2
3	新增 HDMI 大屏（80 柱 + 波形）	新增功能，WNS 仍为正
4	SD 图片背景（双帧缓冲方案）	BRAM: 68.5→100.5（**HDMI 部分 +32 而非 +84**）

这五个阶段反映出一个规律：前两个阶段（同步复位、FFT 架构）是从不能工作到能工作，收益最大（DPOR-1 从 11 降到 0、DSP 从 2048 降到 4）；后三个阶段（HDMI、

SD 背景）是从能工作到能交付，收益体现在细节质量而非核心指标。这也是一个正常的开发节奏，先把功能跑通，再把工程做干净。

2. SD 图片背景的四个方案取舍

表 16　SD 图片背景候选方案对比

方案	存储	每份	份数	总 BRAM	WNS	结论
A	640×360×16	103	2	206	——	超出器件大小
B	640×360×8	52	2	134	+0.011	实测色阶严重，否定
C	320×180×16	26	1	36.5	+1.84	省资源但 LCD 有黑边
D	**480×272×16 + 320×180×16**	**58+26**	**2**	**100.5**	**+1.78**	**采用**

四个方案的取舍反映了一次完整的工程决策过程：

- **方案 A：**是最直接的想法（存两份全图），但一算 BRAM 就发现装不下，所以这一步应该在动手之前算，而不是综合失败之后算。
- **方案 B：**8 位色在资源上可行，甚至能构建成功，但上板后渐变图上的色阶严重到不可接受。
- **方案 C：**只需存 320×180 一份数据，最省资源，但 LCD 上片只占中间一小块、四周是背景色，观感较差。
- **方案 D：**最终胜出，两部分各存一份适合自己大小的数据，节省资源的同时也能满足时序要求。

其中，还有一个细节，为什么没用一个缓冲两个读口？事后发现，Vivado 的 BRAM 推断不支持 1 写 2 读跨时钟，会退化成 LUTRAM 并触发 `DRC UTLZ-1`（需要 86 452 个 `RAMD64E`，而器件只有 19 000 个）。后面即使尝试加上 `(* ram_style = "block" *)` 也无法改变。

4 总结

4.1 大模型协作记录

4.1.1 所用 AI 工具与使用环节

表 17　AI 工具与用途

序号	工具	类别	主要用途
1	pi（编码代理 CLI）	智能体框架	工具调用、长会话、文件读写
2	deepseek-flash（DeepSeek）	主对话模型	设计讨论、RTL 编写、调试、文档
3	exa（MCP）	联网搜索	器件手册、协议细节、第三方实现比对
4	context7（MCP）	库文档检索	Vivado Tcl / Verilator 参数
5	searchcode（MCP）	仓库搜索	找 TMDS 编码器独立实现做交叉验证

表 18　使用环节与具体目的

编号	环节	具体用途
A-01	定点格式设计	Q 格式、位宽预算、舍入与饱和策略
A-02	滤波器结构选择	一阶低架 vs 二阶 peaking、级联顺序、条件数
A-03	FFT 架构权衡	存储式 vs 流式、单蝶形 vs 并行
A-04	跨时钟域方案	格雷码 FIFO vs 双打 vs 电平翻转握手
A-05	显示架构	扫描式渲染 vs framebuffer，参数化边界
B-01	RTL 编写	60 个 Verilog 模块 + 39 个 testbench
B-02	黄金模型	Python 定点模型，逐位对拍
B-03	生成器脚本	旋转因子、频谱分组表、字模、WM8960 寄存器表
B-04	构建流程	`build.tcl` / `program.tcl` / `run_iv.sh`
B-05	调试	读波形、读综合报告、定位时序/资源/DRC 问题
B-06	文档	16 个设计文档 + 825 条问题账本

4.1.2 主要提示方式

本节按使用环节的编号（A-01 到 A-05）各给出一个真实对话案例，每个案例都包含完整的用户提示词与 AI 回答要点。

A-01　定点格式设计

用户提示词

我在做 5 段双二阶 IIR 均衡器，输入是 WM8960 的 24 bit 采样，系数想用 18 bit。请帮我做位宽预算，要求：

1. 每一级的中间结果位宽都要有推导依据，不能只给结论；
2. 说明舍入用四舍五入还是截断，各自对直流偏置的影响；
3. 说明饱和放在哪一级，为什么不在每一级都饱和；
4. 给出一个可以自查的方法，让我能验证位宽确实够用。

AI 回答要点

数据 Q1.23 与系数 Q2.16 相乘得 Q3.39，两个乘积相加再留 1 位给舍入得 41 bit；状态寄存器取 48 bit 留足余量。舍入采用加 2^{15} 再右移 16，即四舍五入，相比截断可以把直流偏置从约 -0.5 LSB 压到 ± 0.5 LSB 以内。饱和只放在最后一级输出，中间级不饱和，原因是逐级饱和会在强信号下产生非线性失真，而最后一级饱和只是限幅。自查方法：用扫频正弦输入，对比 RTL 输出与 Python 双精度参考的最大绝对误差，若小于 1 LSB 即认为位宽足够。

下面是舍入与饱和在 RTL 里的写法，以及与之对应的 Python 参考实现：

```
localparam [AW-1:0] RND = {{(AW-1){1'b0}}, 1'b1} << (SH-1);  // 2^15

wire signed [AW-1:0] accr = acc + RND;      // 先加偏置，再右移
wire signed [AW-1:0] ysh  = accr >>> SH;    // 算术右移，保留符号
```

```
def sat24(v: int) -> int:
    # 饱和到 24 bit signed，对应 RTL 里的 MAXV/MINV 比较
    if v > SAMPLE_MAX:      # +8388607
        return SAMPLE_MAX
    if v < SAMPLE_MIN:      # -8388608
        return SAMPLE_MIN
    return v
```

A-02　滤波器结构选择

用户提示词

5 段均衡器，我打算这样分配：两段低架、一段 8 kHz 的 peaking、两段高架。请评估这个结构：

1. 频段之间是否会有明显的相互影响；
2. 级联顺序对数值条件数有什么影响，建议怎么排；
3. 一阶低架和二阶低架在这种场合哪个更合适。

AI 回答要点

相邻段的中心频率若相差不到一个倍频程，增益会相互影响，建议把相邻段拉开到 1.5 个倍频程以上。级联顺序建议按 Q 值从低到高排列，即低频架在前、peaking 居中、高频架在后，这样中间状态的动态范围最小，最不容易在定点实现中溢出。一阶低架在高频段的衰减斜率只有 6 dB/oct，若只需要粗略的低频提升就够用；二阶可以得到 12 dB/oct 并带 Q 控制，但系数灵敏度更高，需要更宽的位宽。

按上述建议改写后的频段分配如下（Python 端的设计代码，仅保留结构部分）：

```
BANDS_HZ = [100.0, 300.0, 1000.0, 3000.0, 8000.0]   # 相邻段 > 1.5 倍频程
BAND_Q   = 1.0
FS       = 48000.0

def design_band(band_idx, gain_db):
    f0 = BANDS_HZ[band_idx]
    # band0: 一阶低架。二阶低架分母在近 DC 处相位抵消严重，
    #        18 bit 系数不够（实测频响误差 1.4 dB）
    if band_idx == 0:
        return first_order_low_shelf(f0, FS, 10 ** (gain_db / 20))
    # band1~4: 二阶 peaking。DC 与 Nyquist 增益恰为 1，不干扰相邻段；
    #          band4 若用二阶高架，b0 会到 2.12 超出 Q2.16 的 [-2,2) 范围
    return peaking_eq(f0, FS, BAND_Q, gain_db)
```

A-03　FFT 架构权衡

用户提示词

我需要 1024 点 FFT，采样率 48 kHz，器件只有 240 个 DSP48。请对比三种架构并给出选择依据：1. 全并行；2. 4 路并行；3. 单蝶形时分复用。要求每种都算出 DSP 用量、BRAM 用量、整帧耗时，并算出占帧周期的百分比。另外说明：为什么这个百分比比“绝对耗时”更重要。

AI 回答要点

全并行需要 512 个蝶形乘 4 个乘法器共 2048 个 DSP48，超出器件总量 8 倍，直接排除。4 路并行需要 512 个 DSP48，仍然超出 240 的上限。单蝶形需要 4 个 DSP48、2 块 BRAM，整帧 30720 拍即 0.64 ms，占 46.9 ms 帧周期的 1.3%，余量 75 倍。百分比比绝对耗时更重要，因为 FFT 是周期性任务，只要能在下一个帧周期到来前算完就不会丢数据；剩余的时间可以全部让给其它逻辑，而 FPGA 的资源是分时共享的。

单蝶形方案的关键是地址发生器必须用位操作算出蝶形点对，不能有除法或取模，否则会引入 DSP 并恶化时序：

```
// 第 stage 级：把 cnt 的二进制位中间插一位
assign p = ((cnt_x >> stage_w) << sh_p) | (cnt_x & lo_mask);  // 插 0
assign q = p | (ONE << stage_w);                              // 插 1
assign tw_idx = (cnt & lo_mask[LOG2N-2:0]) << sh_tw;          // 因子索引
```

A-04　跨时钟域方案

用户提示词

我有三处跨时钟域：

1. 60 根柱高每根 9 bit，从音频域传到显示域；
2. 十几个配置位（模式、视图、速度），方向同上；
3. SD 卡读启动信号，是一个单拍脉冲，从 48 MHz 传到 25 MHz。

请分别给出推荐的跨域方式，并说明为什么不统一用同一种。特别说明：如果我图省事，全部用两级触发器，会出什么问题。

AI 回答要点

第一处是多比特总线，不能逐位双打，因为各位的传播延迟不同，会出现某些位已经更新、某些位还停留在旧值的“半新半旧”状态，接收侧可能采到一个从未存在过的数值。应当用异步 FIFO，把整帧数据作为一个原子单位传递，本设计用格雷码指针的 `async_fifo`。第二处是慢变的单比特或小向量，用两级触发器即可，延迟两拍对配置生效时间没有影响。第三处是单拍脉冲，如果直接双打，当脉冲宽度小于目标时钟周期时会被整个漏掉，必须先把事件转换成电平状态（翻转一次），再在目标域双打并检测边沿。

三种方式的代码骨架如下。第一种是双打（用于配置位）：

```
(* ASYNC_REG = "TRUE" *) reg [1:0] sync_q;   // 综合器放同一片 SLICE
always @(posedge clk) sync_q <= {sync_q[0], din};
assign dout = sync_q[1];
```

第二种是电平翻转加边沿检测（用于单拍脉冲）：

```
// 源域：把事件变成电平状态
always @(posedge clk_src) if (pulse) tog <= ~tog;

// 目的域：双打后取异或，得到一拍宽的脉冲
always @(posedge clk_dst) sync <= {sync[1:0], tog};
assign pulse_dst = sync[2] ^ sync[1];
```

第三种是格雷码指针（用于多比特总线）。格雷码保证相邻两个值只差一位，采样时最多错一位，不会出现中间态：

```
function [PW-1:0] bin2gray;      // g = b ^ (b >> 1)
```

```
    input [PW-1:0] b;
    begin bin2gray = b ^ (b >> 1); end
endfunction
```

A-05　显示架构

用户提示词

我要同时驱动 LCD 480×272 和 HDMI 640×360，两块屏显示同样的内容，但 HDMI 不要 UI。请对比两种架构：

1. 先渲染到 framebuffer，再分别以两个时钟读出；

2. 渲染核心写成纯组合函数 $(x, y) \to \text{RGB}$，实例化两份。

要求算出方案 1 在 16 位色下的 BRAM 占用，并判断器件是否装得下。另外说明方案 2 在“加新图层”时的代价。

AI 回答要点

方案 1 若以最大分辨率 640×360、16 位色存储，需 230400 像素即约 3.68 Mbit，按每块 BRAM36 可存 36 Kbit 计算需要 103 块，而该器件只有 135 块，加上其它模块必然超出。方案 3 的渲染核心不含任何状态，LCD 侧完全不需要帧缓冲，只有 SD 图片需要真实存储。加新图层时，方案 1 需要在渲染阶段插入并重新计算整个 framebuffer，方案 2 只需新增一个模块并在混色器里排一下优先级，已有逻辑完全不动。代价是方案 2 需要在两个时钟域各跑一份相同的组合逻辑，LUT 用量约为方案 1 的两倍，但本设计 LUT 余量充足。

多柱数映射时要注意：老路径必须用结构而非数学保证零变化。下面的 generate 恒等分支让综合器在 LCD 路径上根本看不到除法，从而保证资源与改动前完全一致：

```
generate
    if (NBARS_DISP == NBARS) begin : g_ident
        assign bbar = dbar[AW-1:0];            // 综合器看不到除法
    end else begin : g_scale
        assign bbar = (dbar * NBARS) / NBARS_DISP;
    end
endgenerate
```

4.1.3 自我纠错轨迹

表 19 给出了智能体自我纠错的五个典型案例：

表 19　自我纠错轨迹

编号	现象	根因	修复 / 沉淀
E-01	改了参数、脚本输出 OK，但上板画面仍居中/颜色错	`str.replace()` 找不到就静默返回原串	加 `assert old in s`；账本 807
E-02	合成报 `DRC UTLZ-1`：需 86 452 个 `RAMD64E`	`mem` 被两个时钟读，导致退化成 LUTRAM	改两份不同尺寸缓冲；账本 789/790/792
E-03	综合报 40 条 `REQP-1839`	`wr2_*` 误放进异步复位块	移出复位块；账本 813
E-04	LCD 与 HDMI 背景不一致	`y[8:3]` 在 360 行时回绕	`YSH = (VDISP>=512)?4:3`；账本 811
E-05	HDMI 图片深蓝显示成暖黄	R/B 互换（LCD 面板是 BGR，反了两次抵消）	HDMI 路径加一次交换；账本 810

4.1.4 智能体工作流设计

整个开发过程不是“让模型生成一段代码然后人工检查”，而是让模型处在一个可以自行验证的闭环里。闭环的核心是三个动作的重复：改代码、跑工具、读报告。其中“跑工具”包含三层检查，任何一层不通过都不允许进入下一层。

1. 第一层是仿真。39 个 testbench 并行跑完约 1 分 30 秒，结果要么全绿，要么明确指出是哪个模块的哪条判据失败。这一层抓的是逻辑错误，包括数值精度、协议时序、跨域采样。
2. 第二层是静态检查。Verilator 补 iverilog 的盲区，能发现多重驱动、组合环、锁存器推断这类仿真跑不出来但上板必炸的问题。Verible 则保证代码风格一致。
3. 第三层是综合与实现。这一层不看代码，只看报告：资源是否超过 80%、时序是否所有时钟对都为正、DRC 是否为零。

三层都过之后才提交并上板。如果任何一层不通过，则根据失败的层级走不同的分支：仿真红说明是逻辑错，需要在本文件内定位（grep 报告、读波形），定位到根因后回到改代码；综合红通常不是 bug 而是架构问题，需要先分析 post_route 的 util 与 timing 报告，判断是资源超限、时序不收敛还是 DRC 违规，再决定是改代码还是改架构。两条分支互不干扰，智能体工作流程如图 12 所示。

图中两条虚线是两种失败的回退路径。左侧红色虚线对应仿真失败，通常可以在当前文件内定位；右侧蓝色虚线对应综合失败，往往意味着要重新审视架构。

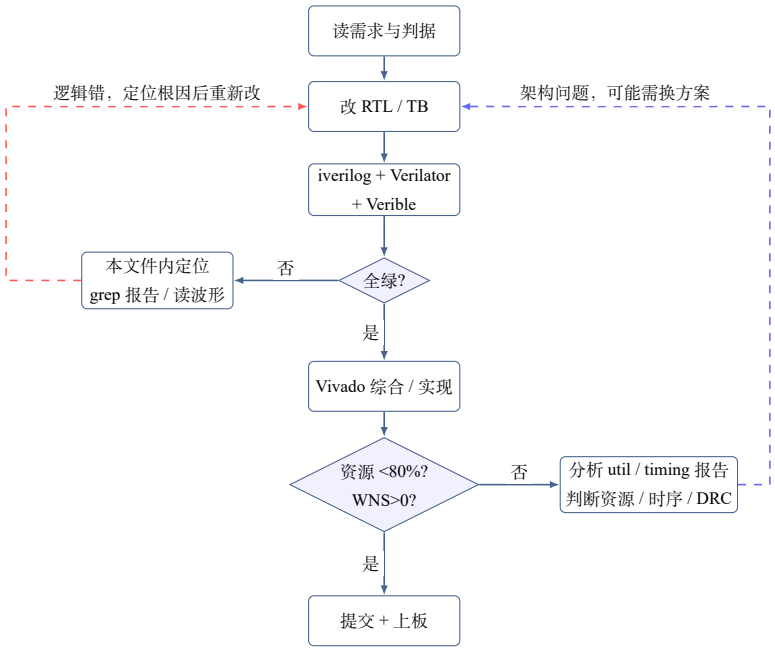


图 12　智能体工作流

4.1.5 采纳、人工修改与核验

针对每个环节 AI 输出结果的采纳情况及人工修改情况的概述如下：

表 20　AI 输出的采纳情况与核验

编号	采纳	人工修改	核验结果
A-01	采纳	无	✓逐位一致
A-02	部分	结构从 4 段改 5 段，加 8 kHz peaking	✓频响误差 <0.5 dB
A-03	采纳	无	✓2048 频点逐位一致
A-04	部分	格雷码 FIFO 改“电平翻转 + 双打 + 边沿检测”	✓4/4 PASS
A-05	部分	加 `WITH_UI/WITH_TEXT/FB_*` 参数族	✓130 560 像素逐点一致
B-01	采纳	多次人工纠错（见 E-01~E-05）	✓39/39
B-02	采纳	无	✓逐位一致
B-03	采纳	`gen_spectrum_map.py` 后参数化	✓60/80 均验证
B-04	部分	Flash 烧写改为“先 `program_hw_devices` 再建 SPI 通道”	✓43 秒成功
B-05	采纳	无	✓WNS 全为正
B-06	采纳	全部数字人工核对	✓与报告一致

核验方法论：1. 逐位对拍；2. 独立真值（`numpy.fft` / `$atan2` / `$sqrt`）；3. 穷举（512 个 bin 分组）；4. 总量守恒；5. 读回验证。

4.2 技能包的提炼过程

4.2.1 三级沉淀体系

表 21　三级知识沉淀

层级	载体	规模	作用
问题账本	`docs/08-skills-index.md`	**825 条**	现象/根因/修复/量化改善，原始可检索
技能包	`.pi/skills/*/SKILL.md`	**3 个**	触发条件/方法/判据/边界，可复用
状态快照	`docs/14-status-snapshot.md`	1 份	一眼看清“能用的/受限的/待办的”

4.2.2 三个技能包的来源、验证与边界

表 22　技能包一览

技能包	来源失败	如何验证有效	边界
`fixed-point-dsp`	EQ 频响差 3 dB（位宽不足）；`WAVE_AMP` 改大反而变小（静默截断）；某频点发散（条件数）	重做 EQ，频响误差 3 dB 改成 0.5 dB；截断问题后续 6 个参数零复发	适用：定点 IIR/FIR/FFT，16~32 bit。不适用：浮点、位宽 <12 bit
`fpga-rtl-flow`	找不到 Vivado；DRC 在 GUI 不报；时序违例方向错；`place` 失败伪装成综合错；构建太慢	初次构建从 9m38s 降至 2m57s；全量仿真从 20m54s 降至 1m30s；后续 20+ 次构建零次返工	适用：纯 RTL、无 block design。不适用：IP integrator、部分重配置
`rtl-verification`	在信号自己更新的沿上读它；非阻塞赋值读不到新值；跨帧/帧内状态共用复位；只测一次漏掉重配路径	39 个 TB 中至少 12 个第一次跑就红；账本里大部分失败是仿真发现不了的，但这几条都是上板必须解决的的	适用：确定性数字逻辑。不适用：模拟/混合信号

4.2.3 边界如何确定

每条技能都必须能回答三个问题：它解决什么现象？它的前提是什么？前提不成立时会怎样？一个典型的反面例子（把特例当通例）：

× **不要写**：“BRAM 一定要加 `(* ram_style = "block" *)`”，并且实测该属性在“1 写 2 读跨时钟”下**无效**。
✓ **应该写**：“BRAM 推断由**访问模式**决定，属性只是建议。1W1R 可靠；1W2R 跨时钟必退化成 LUTRAM，属性也救不了。”

4.3 可扩展之处

表 23　可扩展方向

方向	现状	扩展思路
HDMI 放大平滑	320×180 直接 2× 放大，可见方块边界	加水平双线性：奇数列取 $(p_{sx}+p_{sx+1})/2$，只需 1 个寄存器（方案见 `docs/16`）
HDMI 撕裂	复位后偶发右半屏撕裂	需 ILA 抓 TMDS 或示波器测 5× 时钟相位
外部存储	目前图片存 BRAM（74%）	接 DDR3/SDRAM 可支持 1080p 满帧缓冲；但需手写控制器或引入官方 IP 核以降低工作量
触摸交互	XPT2046 已写驱动，但 LCD 模组无触摸层	换 `TM043NBH02` 即可支持触操作

4.4 心得体会

1. 写 RTL 最难的不是写，在于如何验证

项目初期我以为难点在 RTL 本身。但是做完后才发现，真正耗时的是“怎么知道它是对的”。最后发现一个比较好的做法是给每个模块配一个 Python 定点黄金模型，逐位对拍，比人眼波形来的更精确。

其中印象最深的是在写 `sd_img_load` 模块时：SD 一块是 262 144 字节，而一帧只需 261 120 字节，多出的 1024 字节反复写最后一个地址，会把画面右下角踩坏。而这个 bug 仿真能抓到，但上板难以看出，这就是逐位对拍的价值。

2. 有些资源远比想象中紧

640×360×16 bit 的帧缓冲算下来要 103 块 BRAM，而器件只有 135 块。我以为留 30 块给其它模块够了，但实际发现加上其它器件直接到了 130 块（98%），布线拥塞导致时序崩到 -1.09 ns，`place_design` 直接失败。

之后一共走了三步才得到解决：**1.** 采用 640×360×8（8 位色）：能构建，但 256 色在渐变图上看得到明显色带，被用户否掉；**2.** 采用 320×180 单份（LCD 居中）：省资源，但 LCD 有黑边；**3.** 最后用两份不同尺寸的缓冲：LCD 使用 480×272 全图 + HDMI 做 320×180 的区域裁剪后放大，最终使用 BRAM 100.5 块（74%），时序 +1.78 ns。

3. 能构建 ≠ 能交付

时序违例不会阻止 Vivado 生成 `.bit`，它只会报一个负的 WNS。所以判断标准不能是“构建成功”，而是必须“所有时钟组的 WNS 都为正”。之前因为看到生成了烧录文件就以为成功，差点把一个 WNS $= -0.82$ ns 的比特流烧上板。

4. 调试的三步走

1. **第一级：看报错**，语法检查工具说得清清楚楚的问题不应浪费时间去猜；
2. **第二级：看报告**，综合/实现的 util、timing、DRC、CDC 报告里，工具其实已经把根因写出来了（如“LUT as Distributed RAM over-utilized”直接说明存储器没映射到 BRAM）；
3. **第三级：改设计**，要从根因上修，不能只看症状，同一根因可能在不同模块复发，所以修完还要全量重扫。

5. 关于用大模型写 RTL

这一块给我的体会是：大模型擅长把已知方案写出来，不擅长判断方案对不对、好不好。例如：

- 让它写一个 TMDS 编码器，它写得又快又好，因为这是有标准答案的；
- 让它判断“该用一份缓冲双读口还是两份缓冲”，它会给出一个看起来合理的答案，而真正正确的答案还要靠算 BRAM 数量、去读 UG473、去跑综合验证。

正确的用法是把它当“高速打字机 + 写验证脚本工具”，而不是“决策者”。所有关键决策（架构、位宽、时序预算）必须自己算过、自己验证过。

参考文献

[1] Business Research Insights. *Audio Visualizer Market Size, Share, Growth, and Industry Analysis, By Type (Cloud Based, On-premises), By Application (Android, iOS, Windows) and Regional Insights and Forecast to 2035*. 2026. https://www.businessresearchinsights.com/market-reports/audio-visualizer-market-116451.

[2] Xilinx. *7 Series FPGAs Memory Resources (UG473)*. Xilinx Inc., 2019.

[3] Xilinx. *7 Series FPGAs SelectIO Resources (UG471)*. Xilinx Inc., 2018.

[4] Xilinx. *7 Series FPGAs Clocking Resources (UG472)*. Xilinx Inc., 2018.

[5] Xilinx. *Vivado Design Suite User Guide: Using Constraints (UG903)*. Xilinx Inc., 2020.

[6] 璞致电子科技. 璞致 *Artix-7* 系列之 *PA-Starlite* 开发板用户手册, *V1.0*. 2024.

[7] 璞致电子科技. 璞致 *PA-Starlite* 系列 *FPGA* 开发指南, *V1.2*. 2024.

[8] Wolfson Microelectronics. *WM8960 Stereo CODEC with 1W Class-D Speaker Driver and Headphone Drivers for Portable Audio Applications, Data Sheet Rev. 4.2*. 2013.

[9] Winbond Electronics. *W25Q128JV 3V 128M-bit Serial Flash Memory with Dual/Quad SPI, Data Sheet*. 2019.

[10] A. V. Oppenheim and R. W. Schafer. *Discrete-Time Signal Processing*, 3rd edition. Pearson, 2010.

A 附录：复现说明

1.1 环境要求

表 24　软件环境

工具	版本	说明
Vivado	2020.2	免费 WebPACK 版即可（7A100T 在支持列表内）
iverilog	12.0	`pacman -S iverilog` / `apt install iverilog`
Verilator	5.052	`pacman -S verilator`
Verible	latest	`mason` 或 GitHub release
Python	≥3.10	系统自带

表 25　硬件环境

项	规格
开发板	璞致 PA-Starlite（XC7A100T-2FGG484I）
音频模块	WM8960（接 JM2）
显示屏	璞致 4.3 inch LCD（40 pin，接 JM1）
HDMI 显示器	任意支持 640×360@60 的显示器
microSD 卡	≥1 GB，需要写卡权限（`sudo`）
下载器	板载 Digilent JTAG（TypeC）

1.2 从零开始的完整步骤

1.2.1 步骤 1：获取代码

```
git clone https://github.com/flyingyuya/Sonic-Speed-Parallelism.git
cd Sonic-Speed-Parallelism
```

1.2.2 步骤 2：工具链自检

```
bash scripts/sim/run_iv.sh tools
# 预期：[ok] iverilog / verilator / verible 三行
# 若 Vivado 找不到：export PATH=$HOME/Xilinx/Vivado/2020.2/bin:$PATH
```

1.2.3 步骤 3：全量仿真（约 1 分 30 秒）

```
bash scripts/sim/run_iv.sh
# 预期：通过 39  失败 0  跳过 0    <-- 环境是否正确的黄金标准
# 单项：bash scripts/sim/run_iv.sh fft_core
# 含 tb_top 的全量：FULL=1 bash scripts/sim/run_iv.sh
```

1.2.4 步骤 4：静态检查

```
verible-verilog-lint --rules_config_search $(find rtl sim -name '*.v')
# 预期：无输出（退出码 0）
```

1.2.5 步骤 5：综合与实现（约 9~10 分钟）

```
vivado -mode batch -source scripts/vivado/build.tcl \
       -tclargs xc7a100tfgg484-2 top constrs
# 预期：构建 0  CRITICAL 0  WARNING 0
#       LUT 11716 (18.48%)  BRAM 100.5 (74.44%)
#       clk_sys +1.779ns    clk_hdmi +4.028ns
# 只综合（约 3 分钟，看资源够不够）：
SYNTH_ONLY=1 vivado -mode batch -source scripts/vivado/build.tcl \
       -tclargs xc7a100tfgg484-2 top constrs
```

1.2.6 步骤 6：准备 SD 卡（需要 sudo）

```
lsblk -o NAME,SIZE,TYPE,TRAN | grep -E "sd|mmc"     # 找到读卡器，如 sda

# 先 dry-run（强烈建议）
python3 scripts/sdcard/mkimage.py --dry-run /dev/sda img1.png img2.png img3.png
# 预期每张图 261120 字节 = 480x272x2  <-- 数字对了就说明格式正确

# 正式写入（脚本内建系统盘保护）
echo YES | sudo python3 scripts/sdcard/mkimage.py /dev/sda img1.png img2.png img3.png
```

1.2.7 步骤 7：上板

```
# 1) 验证 JTAG
vivado -mode batch -source scripts/vivado/program.tcl -tclargs detect
# 预期：发现 JTAG 目标 ... 认到目标器件 xc7a100t

# 2) 下载到 SRAM（10 秒，掉电即失，开发期用这个）
vivado -mode batch -source scripts/vivado/program.tcl

# 3) 固化到 Flash（43 秒，掉电保持，定型后才用）
vivado -mode batch -source scripts/vivado/program.tcl -tclargs flash
# 预期：Mfg ID ef / Erase-Program-Verify successful
# 注意：JTAG 优先于 QSPI，验证 Flash 启动必须拔掉 JTAG 线再上电
```

1.3 常见问题

表 26　常见问题与解决

现象	原因	解决
`run_iv.sh` 全失败	iverilog 太老或文件清单引用了已删文件	需 ≥11.0；`grep -rn <文件名>` 扫一遍清单
综合报 `UTLZ-1`（LUTRAM 超限）	存储器多端口访问模式不匹配推断模板	复制实例，不要加读口
上板 LCD 全白/全黑	LCD 时序参数错	跑 `run_iv.sh lcd_timing`
SD 图片乱码/斜切	卡里数据尺寸与 RTL 的 W/H 不匹配	重新写卡，确认每图 261 120 字节
HDMI 颜色偏黄	R/B 通道互换	LCD 面板是 BGR，HDMI 路径需再换一次
HDMI 复位后偶发右半屏撕裂	已知未解决（TMDS 层）	重按复位可恢复；需 ILA/示波器